

Earlier DSPL–DDQN Study

Anonymous Author(s)

Abstract. Microservice-based systems (MSS) operate in dynamic environments, where contextual changes can degrade Quality of Service (QoS) and affect user experience (UX). While self-adaptive approaches can mitigate these challenges, existing solutions often focus on infrastructure level mechanisms, such as autoscaling, which provide limited support for reasoning over application-level configuration alternatives and their QoS–UX trade-offs. This paper presents an application-level self-adaptation approach that combines Dynamic Software Product Lines (DSPL) with Deep Reinforcement Learning (DRL). DSPL defines the valid configuration space of the MSS, while a Double Deep Q-Network (DDQN) agent learns workload-aware reconfiguration decisions using a performance prediction model (PPM) as a surrogate environment. The proposed approach is evaluated on CaseStudy-A, a real microservice-based case study, through trace-driven workload emulation and compared against a genetic algorithm (GA) baseline and an oracle upper bound. Results show that the approach achieves near-oracle decision quality and better decision quality than a budgeted GA baseline on the same evaluation states. Under trace-driven emulation with workload stress and recovery periods, the DDQN agent protects end-to-end latency during stress while maintaining competitive UX under relaxed workload. The results also show that DDQN enables substantially faster runtime decision-making than GA under the same per-trigger budget, supporting timely adaptation in dynamic MSS environments.

Keywords: Self-Adaptive Systems · Microservice-based Systems · Dynamic Software Product Lines · DDQN · Quality of Service · UX.

1 Introduction

Microservice-based systems (MSS) have emerged as a prominent architectural style for developing modern software applications. Rather than structuring an application as a monolith, this style decomposes functionality into small, loosely coupled, independently deployable services. This modular organization improves maintainability, supports rapid evolution, and enables flexible deployment and scaling in distributed environments. Consequently, MSS are increasingly adopted in diverse domains, including web and mobile applications, cyber-physical systems, and Internet of Things (IoT) systems. Yet MSS typically operate in highly dynamic environments: sudden shifts in workload, usage patterns, and resource

availability are common and cannot be fully anticipated due to design-time uncertainty. As a result, maintaining Quality of Service (QoS) across fluctuating operating conditions remains challenging.

A second challenge concerns application-level trade-offs that directly affect user experience (UX) delivered through application functionality [26]. To illustrate this, consider our case-study system CaseStudy-A, a microservice-based website where users browse and purchase books. In this system, UX-enhancing services refer to optional or richer microservices/variants, such as personalized recommendations, detailed reviews, or richer content presentation, which improve the user experience but are not always critical for the core transaction. Under high workload e.g., flash-sale-like conditions, keeping all these services active may increase processing and communication overhead, leading to higher end-to-end latency and possible Service Level Objectives (SLO) violations. We therefore focus on response time as the operational QoS metric, since it reflects the user-perceived delay of a request in latency-sensitive MSS. The goal is thus not only to reduce latency, but also to preserve UX-enhancing functionality when workload conditions allow it.

Self-Adaptive Systems (SAS) have emerged as a promising solution to these challenges. Equipping MSS with self-adaptation capabilities enables them to dynamically adjust their behavior under varying workloads and operating conditions. However, most existing MSS self-adaptation approaches still rely primarily on infrastructure-level mechanisms, such as horizontal or vertical auto-scaling and microservice placement, typically driven by rule-based engines [25,18,5]. Although these mechanisms are effective for resource management, they have two fundamental limitations. First, they are resource-bounded: when resources are saturated or quotas are reached, scaling actions become ineffective or fail, yet performance violations may persist [1,21]. Second, autoscaling primarily changes resource allocation, not the application-level functionality; it does not directly decide which UX-enhancing microservices should be enabled or disabled to keep QoS within bounds. As a result, infrastructure-centric adaptation may be insufficient to maintain acceptable QoS under sustained stress, and it does not provide a principled mechanism for negotiating the QoS–UX trade-off at runtime.

To address these challenges, this study adopts an application-level self-adaptation strategy that builds on an earlier anonymized study of Dynamic Software Product Line (DSPL)-based adaptability in MSS [4]. In this approach, an MSS is modeled as a configurable set of microservices, explicitly capturing variability across both functional and non-functional features. To select suitable configurations at runtime, we integrate the DSPL framework with a Deep Reinforcement Learning (DRL) agent [24], specifically a Double Deep Q-Network (DDQN). The DDQN agent is designed to balance QoS objectives and user experience under dynamic workload conditions. In this paper, QoS is operationalized using end-to-end response time (RT), i.e., latency, as the primary QoS metric. By combining structured variability modeling with data-driven decision-making, the proposed approach supports runtime selection of configurations that better balance the RT–UX trade-off.

This paper presents the proposed DSPL-DRL approach and evaluates it on a real microservice-based case study (CaseStudy-A) under trace-driven workloads. The results show that the approach makes configuration decisions that closely match the oracle and achieves better decision quality than a budgeted GA baseline on the same evaluation states. Under trace-driven emulation with workload stress and recovery periods, the DDQN agent protects end-to-end latency during stress while maintaining competitive UX under relaxed workload. The results also show that DDQN enables substantially faster runtime decision-making than GA under the same per-trigger budget, supporting timely adaptation in dynamic MSS environments.

2 Related Work

We review prior work relevant to our study in two main research streams: (i) self-adaptation in microservice systems (MSS), and (ii) DSPL-based self-adaptation. Table 1 compares these works in terms of adaptation scope, supported actions, use of explicit variability models, and decision-making mechanisms.

MSS self-adaptation. Research on MSS self-adaptation covers a broad range of scopes and techniques. Some studies extend orchestration frameworks with explicit adaptation actions and architectural support. For instance, Zhang et al. [27] extend Kubernetes with rule-based actions such as adding, deleting, or upgrading services, while Liu et al. [15] propose a multi-layer reference architecture that separates infrastructure-level scaling from higher-level application adaptation. However, both mainly rely on predefined rules or architectural guidance, without systematically constraining application-level decisions through an explicit variability model. Other works move toward service selection based on user needs, as in Santi’c/De Sanctis et al. [7,8], but the focus remains on service composition rather than runtime reconfiguration of a predefined configuration space. He et al. [12] support application-level refactoring actions, such as switching, splitting, and merging microservices, and combine heuristic reasoning with GA-based search under SLA constraints, which may introduce search overhead at runtime. In parallel, another line of work emphasizes infrastructure elasticity, including improved autoscaling control in Kubernetes [1], learning-based threshold policies for scaling [20], and runtime microservice evolution to cope with workload shifts [19]. Overall, MSS self-adaptation research either focuses on infrastructure-level elasticity or considers application-level changes without using an explicit DSPL/feature model to constrain valid runtime reconfigurations.

DSPL-based self-adaptation. DSPL-based approaches explicitly use feature and variability models to support safe runtime adaptation. Alf  rez et al. [2] employ DSPL models to guide service composition reconfiguration under SLA triggers. Mousa et al. [17] combine DSPL workflow variability with multi-objective optimization and QoS prediction. Metzger et al. [16] integrate DSPL with reinforcement learning and feature-model-guided exploration to address large discrete configuration spaces. In contrast, our approach provides a fully imple-

Table 1: Comparison of related work

Work	Scope	Action	Objective(s)	FM	Decision-making
Zhang et al. [27]; Liu et al. [15]	App./orch. level	Rule-based service actions; architectural guidance	QoS, RT/SLA	No	Rules/architecture
Santić / De Sanctis et al. [7,8]	Composition	Service selection/composition	User requirements + partial QoS	No	Goal/model driven
He et al. [12]	App. level	Switch/split/merge services	QoS with SLA constraints	No	Heuristic + GA
Ahmad et al. [1]	Infra. level	Autoscaling/resource management	QoS + efficiency	No	Controller/heuristic
Rossi et al. [20]	Infra. level	Threshold-based scaling	QoS + resource metrics	No	RL-based policy
R.-Filho et al. [19]	Service impl.	Runtime microservice evolution	QoS under workload shifts	No	Learning driven
Alfárez et al. [2]	DSPL composition	DSPL-guided reconfiguration	SLA/QoS	Yes	Rule/model driven
Mousa et al. [17]	DSPL workflow	Context-aware reconfiguration	QoS + cost	Yes	NSGA-II
Metzger et al. [16]	DSPL cloud configs	Runtime configuration selection	QoS(RT/energy)	Yes	Online RL
This Study	App.-level DSPL	Configuration/variant switching	Workload-aware RT-UX trade-off	Yes	DDQN + GA baseline

mented application-level DSPL adaptation for MSS, where the agent selects valid microservice configurations and variants under observed workload, guided by a workload-aware RT-UX trade-off, and is evaluated against a budgeted GA baseline.

3 The Proposed Approach: Overview

The proposed approach combines DSPL and DRL to enable intelligent self-adaptation in MSS. DSPL defines the valid adaptation space as a set of configurations with different QoS and UX characteristics, while the DRL implemented as DDQN agent learns to select the configuration that best fits the current context. By combining structured variability modeling with adaptive decision-making, the approach supports QoS-compliant and workload-aware adaptation under dynamic conditions. The following subsections describe the DSPL and DRL components in detail.

3.1 Dynamic Software Product Line Feature Model

From a DSPL perspective, a running MSS can be viewed as a collection of features that can be dynamically enabled or disabled in response to changes in the operating context. DSPL employs a runtime variability model, most commonly a feature model (FM) that captures the variation points and the constraints among features [11]. Currently, our approach supports two types of variability points: *alternative* (XOR), where exactly one variant (feature) is selected from a set of alternatives, and *optional*, where a feature may be either included or omitted.

In our design, each feature in the FM, particularly the leaf features, is uniquely mapped to a specific microservice implementation within the system.

This mapping establishes a clear one-to-one correspondence between the feature model and the deployed architecture. The DSPL component in our work is responsible for realizing runtime variability by generating valid combinations of microservices that conform to the constraints of the FM. Each combination represents an alternative system architecture, or more precisely, a distinct configuration of the MSS.

Variability at the user-experience level is defined at the granularity of individual microservices. In the current approach, we associate each concrete feature (microservice variant) with a UX contribution value that reflects its expected impact on the end-user experience. At design time, the software engineer can estimate these UX contributions based on domain knowledge (e.g., whether a variant provides richer content, more accurate recommendations, or more detailed feedback). These contributions are represented as mean-value estimates and are later aggregated to compute a configuration-level UX score, which is used by the decision-making component. While this paper uses static UX contribution estimates for our case study, as described later, the same formulation can be instantiated using real user-interaction signals when available. For example, UX contributions can be derived from operational logs using metrics such as click-through rate (CTR) and dwell time [26], either directly or through a learned mapping that relates observed engagement to the corresponding microservice variants.

We illustrate our approach using the *CaseStudy-A* MSS case study. Figure 1 shows the FM of this system, while Table 2 lists the system features, the names of their variants, and their descriptions. The last column of the table provides static values at the variant level for end-user experience (*UX*). Consider the two valid configurations of *CaseStudy-A* highlighted in Figure 1¹. From the figure, Configuration X attains a higher total UX score S_{ux} because it includes features with strong positive impact on user satisfaction, such as the *Accurate Recommender* and *Detailed Review* services. In contrast, Configuration Y attains a lower S_{ux} by selecting lighter variants, such as a *lower-quality recommender* and a *Review Summary* service, which may reduce user interactions but can help protect response time under high workload.

Alongside these UX-based scores, each configuration is also associated with configuration-level QoS attributes, particularly response time (RT), as described later in Section 4. By jointly modeling UX levels and QoS attributes for each configuration, the DSPL component of our approach defines a structured adaptation space that captures both functional and non-functional trade-offs. Selecting the most suitable configuration under dynamic conditions requires an intelligent

¹ The depicted configurations respect all FM constraints. For example, in an XOR relationship, selecting a variant implies that its parent feature is also selected. In each configuration, an active (selected) feature is indicated with a binary value of 1, whereas an inactive (unselected) feature is assigned 0. The *user experience score* S_{ux} of a configuration is obtained by summing the *UX* contributions of all its active features.

Table 2: Details of microservices and variants in the CaseStudy-A FM

Service	Variant	Description	UX
DisplayBook (DB)	DB-Text	Text-based display with rich details, faster loading in low-bandwidth settings, and accessibility support.	0.7
	DB-Image	Image-based display with stronger visual appeal and marketing potential.	0.2
Inventory (Inven)	Inven-Light	Lightweight inventory service showing basic stock information.	0.3
	Inven-Enhanced	Enhanced inventory service showing richer details, e.g., stock count, best-selling items, and notifications.	0.8
Recommender(Recom)	Recom-Light	Lightweight recommender with lower accuracy.	0.2
	Recom-Accurate	Personalized recommender with higher accuracy using advanced algorithms.	0.8
Review (Rev)	Rev-Summary	Top-five reviews or a short review summary for faster display.	0.2
	Rev-Detailed	Richer review presentation with filtering/personalization, requiring more computation.	0.7
Advertisement	–	Promotional or book-related advertisements.	0.1

reasoning mechanism; this is provided by the DRL component, which performs data-driven decision-making over the DSPL-defined adaptation space.

3.2 Deep Reinforcement Learning-Based Decision Making

A self-adaptive MSS relies on an adaptation management component capable of monitoring the target system and its external context, analyzing changes, planning suitable adaptation actions, and applying them. These steps follow the well-known MAPE (Monitor-Analyze-Plan-Execute) reference model proposed in autonomic computing paradigms [14]. In our approach, the Analyze and Plan phases are driven by a trained DDQN agent. The agent is trained offline using an offline dataset together with a performance prediction model (PPM) that estimates the end-to-end response time (RT) of candidate configurations under the observed context. In general, the context in MSS can include operating-environment conditions, system-level signals from the managed microservices, and user-behavior indicators. In our current implementation, however, the context is primarily captured by the external workload experienced by the system.

To illustrate how the basic DDQN components are mapped to our approach, we formalize them as a Markov Decision Process (MDP) defined by the tuple (S, A, R, T) :

- S : The set of states, combining the external workload context (requests per minute RPM) with internal system performance (end-to-end response time RT), represented as:

$$\text{State} = [RPM, RT]$$

- A : The set of actions, corresponding to configuration indices generated by the DSPL component, where each index represents a unique configuration.
- $R : S \times A \rightarrow \mathbb{R}$: The reward function, designed to promote configuration decisions that protect RT under high workload while preserving UX when workload is relaxed. Reward details are shown in the next section.
- $T : S \times A \rightarrow S'$: The transition function, describing how the system state evolves as the workload progresses over the trace and as the selected configuration affects the predicted response time.

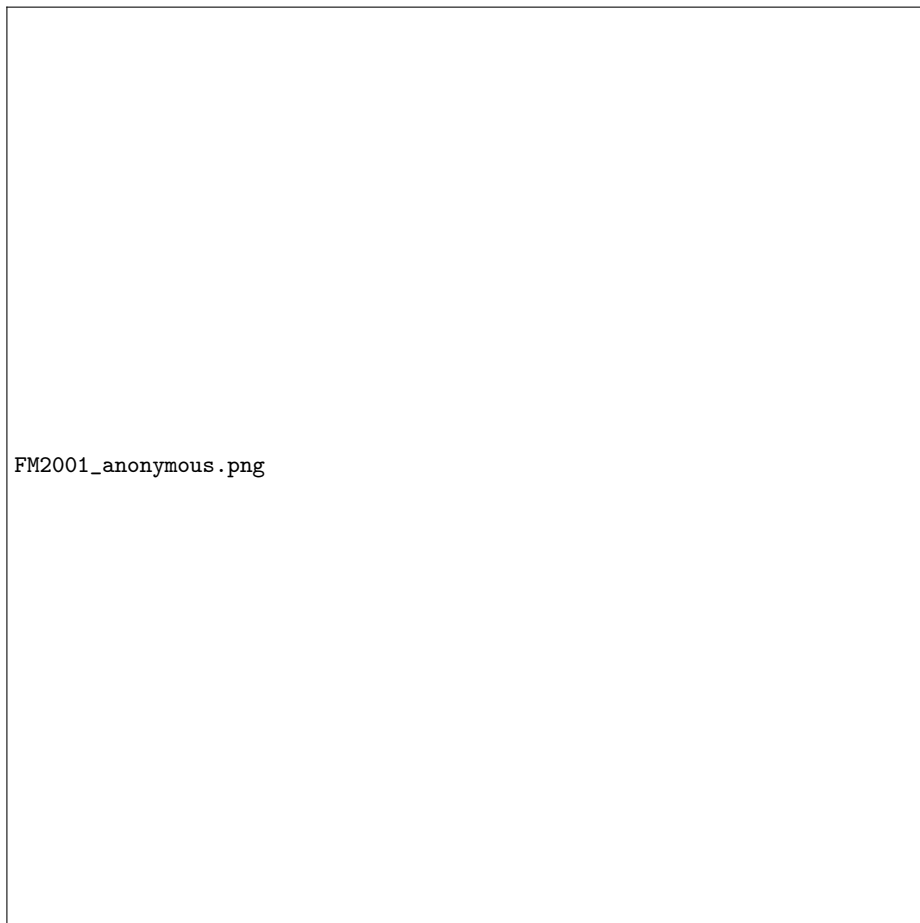


Fig. 1: Feature Model (FM) of the anonymized CaseStudy-A MSS with example valid configurations X and Y

Using this formalism, the agent learns a policy $\pi : S \rightarrow A$ that adapts configuration decisions to the current workload conditions. The policy aims to protect QoS during high-load periods by reducing response time, while preserving higher UX when the workload is relaxed, according to the workload-aware reward design.

3.3 Reward Function: Workload-Aware RT–UX Trade-off

The reward function reflects a workload-aware trade-off between RT protection and UX. Under high workload, the reward emphasizes minimizing RT to keep the system within the SLO threshold τ . Under relaxed workload, it shifts emphasis toward preserving UX by favoring configurations with higher UX value.

At each control interval, the agent selects a valid DSPL configuration as an action a and receives a reward as represented in equation below:

$$R = -\omega(\text{RPM}) \cdot \widehat{RT} + (1 - \omega(\text{RPM})) \cdot \widehat{S}_{ux}(a) - \lambda_{SLO} \mathbb{I}[RT > \tau]. \quad (1)$$

where

- R is the scalar reward.
- $\widehat{RT}$ is the observed end-to-end response time under configuration a .
- $\widehat{RT}$ is the normalized response time.
- $\widehat{S}_{ux}(a)$ is the UX value of configuration a , computed as:

$$\widehat{S}_{ux}(a) = \sum_{i=1}^n UX_i \cdot f_i(a), \quad f_i(a) \in \{0, 1\},$$

where n is the number of features in the feature model, UX_i is the utility contribution of feature i (higher is better UX), and $f_i(a) = 1$ if feature i is enabled in configuration a , and 0 otherwise.

- $\widehat{S}_{ux}(a)$ is the normalized UX utility (normalization reported in the experimental settings).
- RPM is the observed request rate (requests per minute).
- $\omega(\text{RPM}) \in [0, 1]$ is a workload-dependent trade-off weight (Eq. (2)).
- τ is the response-time SLO threshold (ms).
- $\mathbb{I}[RT > \tau]$ equals 1 if the SLO is violated and 0 otherwise.
- $\lambda_{SLO} > 0$ is a constant penalty applied when the SLO is violated.

Workload-aware weight. We compute the trade-off weight from the current request rate:

$$\omega(\text{RPM}) = \text{clip}\left(\frac{\text{RPM} - \text{RPM}_{low}}{\text{RPM}_{high} - \text{RPM}_{low}}, 0, 1\right). \quad (2)$$

where

- RPM_{low} is a relaxed-load anchor.
- RPM_{high} is a high-load anchor.
- $\text{clip}(\cdot, 0, 1)$ bounds the value to $[0, 1]$.

Finally, $\omega(\text{RPM}) \rightarrow 1$ as the workload approaches RPM_{high} , increasing the emphasis on RT minimization, while $\omega(\text{RPM}) \rightarrow 0$ as the workload approaches RPM_{low} , increasing the emphasis on UX preservation.

4 Operational Framework of the Approach

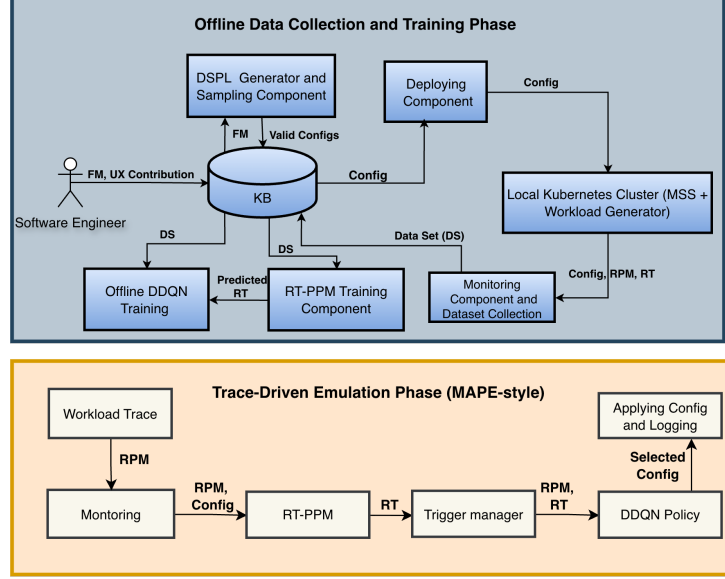


Fig. 2: Operational Framework of the DSPL-DRL Approach.

The operational framework instantiates the DSPL and DRL components of the proposed approach as a complete self-adaptation workflow for MSS. It comprises two tightly connected phases: an offline data collection and training phase, where the PPM and DDQN policy are learned, and a trace-driven emulation phase, where the learned policy is evaluated under evolving workload conditions through emulated configuration switching. The following subsections describe these phases in detail.

4.1 Offline Data Collection and Training Phase

This phase aims to construct a surrogate training setting that reflects the behavior of the target MSS under dynamic workload conditions. As illustrated in Figure 2, the process begins with a software engineer populating the approach’s knowledge base (KB) with the FM of the MSS and the UX contributions of its microservices/variants. This initial knowledge establishes the adaptation space and links each valid configuration to a corresponding UX value.

A fundamental element of this phase is the DSPL component, which interprets the FM to generate the set of valid configurations. Since the full adaptation space may be large, a sampling technique can be used to select a representative subset of configurations for data collection. The sampled configurations are then

executed under trace-driven workloads, and the monitoring component records key QoS properties (e.g., RT in our case) for each deployed configuration.

The collected measurements form an offline dataset that captures how the MSS behaves across the sampled adaptation space under varying workload levels. This dataset becomes the foundation for training a performance prediction model (PPM) [23] that generalizes system behavior to unseen configuration–workload pairs. By integrating the trained PPM into a dedicated surrogate environment, the approach enables the DDQN agent to explore a broad range of configuration decisions without incurring the cost, risk, or delay of live executions. Within this safe but representative environment, the agent learns a policy that maximizes long-term reward by balancing predicted RT with UX values according to the workload-aware reward design.

4.2 Trace-Driven Emulation Phase

In the trace-driven emulation phase, the DDQN agent is evaluated by replaying a workload trace and emulating configuration switching decisions over time. This phase is designed to approximate the behaviour of runtime MAPE loop [14] in a controlled and repeatable setting: instead of monitoring and reconfiguring a live deployment, we iterate over trace steps (e.g., minutes), observe the current workload (RPM), and estimate the RT of the currently active configuration using the trained PPM. This provides a lightweight proxy for the Monitor and Analyze steps, while enabling the Plan step to invoke the DDQN policy whenever an adaptation trigger is met.

When a trigger occurs, the DDQN policy selects a new configuration based on the current workload–RT state. The selected configuration is then applied within the emulation by updating the active configuration identifier, and subsequent trace steps proceed from this updated setting. Throughout the replay, we log the workload level, the selected configuration, the predicted RT, the corresponding UX value, and the achieved reward, producing a complete time series that supports subsequent analysis of RT protection under stress periods and UX preservation under relaxed periods. The emulation includes separate triggers for RT protection and UX upgrading, which are defined in Section 5.2.

5 Evaluation and Experiments

To evaluate the effectiveness of the DSPL–DRL approach, we define the following research questions (RQs):

- **RQ1:** *How close is the learned DDQN policy to the oracle, compared with a budgeted GA baseline, on the same evaluation dataset?*
- **RQ2:** *Under trace-driven emulation, can the learned DDQN policy balance RT protection and UX preservation, compared with a budgeted GA baseline?*
- **RQ3:** *How fast is runtime decision-making for DDQN compared with GA under the same per-trigger budget?*

5.1 Experiments Setup

The evaluation was conducted on the CaseStudy-A MSS introduced in Section 3 and implemented in Java Spring Boot. The associated replication link is omitted from this anonymized manuscript. CaseStudy-A was designed and developed to expose explicit DSPL variability, including optional and alternative microservice variants, a capability not commonly supported by existing MSS benchmarks. This makes it suitable for evaluating application-level configuration adaptation and for studying clear RT-UX trade-offs. Using the feature model shown in Figure 1 and the Sat4j reasoner [22], CaseStudy-A has a total of 72 valid configurations. To reduce deployment effort, we used 3-wise sampling with SPLCATool [13] to select 24 representative configurations. The 24 sampled configurations were used only for offline measurement and PPM training, whereas DDQN, GA, and the oracle operate over all 72 valid DSPL configurations. The configuration sample was then deployed on a local Kubernetes cluster running on a Windows machine with a 2.4 GHz CPU and 12 GB RAM. For data collection, a compressed FIFA 1998 World Cup workload trace [3] was replayed using k6 load-testing tool [9], while end-to-end RT was recorded at one-minute granularity. The resulting dataset, collected in approximately 12 hours, was used to train an XGBoost-based PPM [6], selected for its strong predictive accuracy and efficiency in configurable systems [10]. The PPM achieved $R^2 = 0.8917$ on a held-out split and was shared by both DDQN and the GA baseline for fair comparison. The DDQN hyperparameters were selected as the best-performing setting among multiple experimental configurations, as summarized in Table 3. Training was performed on a Mac device with 8 GB RAM and an Apple M2 chip (8-core CPU, 3.49 GHz maximum clock speed), and the learned DDQN policy was finally assessed through trace-driven emulation on the same PPM-based environment.

Table 3: Key parameters used in the reward formulation (Eqs. (1)–(2)) and DDQN training setup

Parameter	Description	Value
τ	Response-time SLO threshold in the indicator term $\mathbb{I}[RT > \tau]$ (Eq. (1)), defined as $P_{0.95}(RT)$ in the offline DS.	4600 ms
τ_{low}	Relaxed threshold used in the trigger logic (e.g., Upgrade-UX), defined as 0.8τ .	3680 ms
λ_{SLO}	Constant penalty when the SLO is violated in Eq. (1).	1.2
RPM_{low}	Relaxed-load anchor in $\omega(RPM)$ (Eq. (2)), set to the 10th percentile of the observed RPM distribution in the training dataset, $P_{0.10}(RPM)$.	388
RPM_{high}	High-load anchor in $\omega(RPM)$ (Eq. (2)), set to the 90th percentile of the observed RPM distribution in the training dataset, $P_{0.90}(RPM)$.	752
$\text{norm}(\cdot)$ (RT)	Normalization used for $\overline{RT}$.	RunningMeanStd
$\text{norm}(\cdot)$ (UX)	Normalization used for $\hat{S}_{ux}(a)$.	Min-max
N	Protect persistence Number of consecutive minutes with $RT > \tau$ required to trigger RT-protection adaptation	2 min
M	Upgrade persistence Number of consecutive minutes with $RPM \leq RPM_{low}$ and $RT \leq \tau_{low}$ required to trigger UX upgrade adaptation	2 min
W / Cooldown window	Number of minutes during which no new adaptation trigger is allowed after a reconfiguration, to prevent oscillations.	3 min
η (learning rate)	Learning rate used to train the Q-network (DDQN).	1×10^{-4}
γ	Discount factor in DDQN.	0.90
Optimizer	Optimizer used for DDQN training.	Adam
Total training steps	Maximum number of offline DDQN training steps.	120,000
Start ϵ	Initial exploration rate in ϵ -greedy during offline training.	1.0
End ϵ	Final exploration rate after decay in offline training.	0.05
Decay steps (ϵ)	Steps over which ϵ decays linearly in offline training.	50,000

5.2 Evaluation

We now report the experimental findings of our DSPL–DRL approach, organized around the research questions defined earlier.

RQ1. Decision quality against the oracle. For RQ1, we assess the decision quality of the learned DDQN policy and a budgeted GA baseline by comparing both methods against the optimal oracle reference. We used an evaluation dataset of 100 points sampled from the original offline dataset. Each evaluation point corresponds to a workload–performance state $s = \langle RPM, RT \rangle$ for the currently active configuration under that workload.

The evaluation dataset is constructed to explicitly include two categories of trigger-relevant states. First, *Protect-SLO* points represent stress conditions, selected under high workload (near/above the 90th Percentile $P90(RPM)$) and high RT ($RT \geq \tau$), where τ is set to the dataset-derived $P95$ RT threshold ($\tau = 4600$ ms) as shown in Table 3. Second, *Upgrade-UX* points represent relaxed conditions, selected under low workload ($RPM \leq P10(RPM)$) and low RT ($RT \leq 0.8 \cdot \tau$), i.e., 3680 ms. In addition, we include a small number of mid-range points (between $P10$ and $P90$) to avoid restricting the evaluation to extremes, while maintaining emphasis on these two categories.

To obtain the *oracle* decision for each evaluation state, we perform an exhaustive brute force search over all DSPL-derived configuration set A and select the configuration that maximizes the same reward function used by the agent:

$$a^*(s) = \arg \max_{a \in A} R(s, a).$$

This oracle acts as an upper-bound reference under our reward definition and the considered configuration space.

We report two complementary metrics. *Match rate* measures the percentage of evaluation states for which the agent selects the same configuration as the oracle. *Regret* quantifies the loss when the agent deviates from the oracle, computed per state as $\Delta(s) = R(s, a^*) - R(s, a_{\text{agent}})$ and summarized over all 100 states. Together, these metrics capture both how often the DDQN reproduces oracle decisions and how costly the remaining mismatches are, particularly in stress states near τ and relaxed states eligible for UX upgrading.

Table 4 summarizes the results. DDQN achieves a higher oracle match rate (94%) than the budgeted GA baseline (90%), indicating that the learned policy more consistently selects the same configurations as the oracle under the same evaluation states. Moreover, DDQN yields a lower mean regret (0.0001 vs. 0.0002), showing that when mismatches occur, the resulting reward loss is smaller. Although GA optimizes the same workload-adaptive reward (including the SLO penalty), it is a stochastic search under a limited budget and may stop at near-optimal configurations, which explains its lower match rate and higher regret compared to DDQN.

RQ2. Balancing RT protection and UX preservation. For RQ2, we compare the learned DDQN policy against the budgeted GA baseline using a trace-driven

Table 4: Decision quality against the oracle on evaluation dataset (100 states).

Method	Oracle Match rate	Mean regret
DDQN	94.00%	0.0001
GA	90.00%	0.0002

Table 5: Trace-driven emulation results under workload stress and recovery

Metric	DDQN	GA
SLO triggers	1	1
Upgrade-UX triggers	1	1
Mean RT (ms)	1921.59	1951.62
Mean UX	0.6948	0.7620

emulation that includes explicit workload stress and recovery periods. Over the 35-minute trace, the controller triggered one RT protection adaptation during stress and one UX upgrade adaptation during relaxed conditions, reflecting its dual operational objectives.

In the trace-driven emulation, the DDQN agent is invoked only when an adaptation condition is met. We use two categories of triggers (with a cooldown to prevent oscillations): (i) a *protect* trigger, raised when the predicted response time of the active configuration exceeds the SLO threshold τ for a set of N consecutive minutes; and (ii) an *upgrade UX* trigger, raised in a relaxed regime when $RPM \leq RPM_{low}$ and the predicted response time remains below τ_{low} for M consecutive minutes. Table 3 shows the values derived for these parameters. When either trigger fires, the emulator queries the learned DDQN policy using the current state $\langle RPM, RT \rangle$ and applies the selected configuration for subsequent trace steps, subject to a cooldown window. Table 5 summarizes the results. Both DDQN and the budgeted GA exhibit stable behavior, requiring only two configuration switches over the entire trace, indicating that the selected configurations remain effective until the workload regime changes. In terms of RT, DDQN achieved a slightly lower mean response time than GA (1921.59 ms vs. 1951.62 ms), suggesting slightly stronger RT stabilization under fluctuating workload. In the relaxed regime (low RPM), GA achieved higher average UX than DDQN (0.7620 vs. 0.6948), indicating a more aggressive UX preference when slack was available. Nevertheless, DDQN maintained competitive UX while preserving lower average RT. Overall, these results indicate that the proposed DDQN policy provides a balanced RT-UX trade-off in trace-driven emulation, with performance comparable to the budgeted GA baseline while relying on fast policy inference.

RQ3. Decision-making time (DDQN vs GA). To answer RQ3, we measure the time required to select the next DSPL configuration once a trigger occurs in the trace-driven emulation. The GA baseline uses a practical budget of $pop=20$ and $gens=10$, chosen to keep per-trigger latency low, resulting in at most 200 candidate evaluations per trigger, while DDQN selects a configuration using a single forward pass (policy inference). We report the mean and median decision time across 100 decision points.

Table 6: Decision-making time per trigger on evaluation dataset (100 states)

Method	Mean (ms)	Median (ms)
DDQN (policy inference)	0.0670	0.0624
GA (budgeted search)	31.3565	31.6481

Table 6 shows that DDQN is below 0.1 ms per decision, while GA requires about 31 ms. This difference is expected because DDQN shifts exploration offline and uses one runtime policy inference, whereas GA searches online at each trigger, evaluating up to 200 candidates with PPM-based RT prediction and reward computation. Thus, DDQN better supports timely adaptation when triggers are frequent.

Summary of the findings. Overall, the results indicate that the proposed DSPL–DRL approach achieves strong decision quality, balanced RT–UX behavior in trace-driven emulation, and clear runtime efficiency advantages over the budgeted GA baseline. These findings support the feasibility of application-level DSPL–DRL adaptation for dynamic MSS.

Threats and Limitations. The findings should be interpreted with some limitations. First, both offline training and trace-driven evaluation rely on a PPM-based surrogate environment, so prediction errors may affect the learned policy and the GA baseline, although the same validated PPM ($R^2 = 0.8917$) was used for fair comparison. In addition, the evaluation focuses mainly on response time and configuration-level UX scores, which do not capture all QoS dimensions or real user behavior. Finally, the study is based on a single case study, a sampled subset of configurations, and one workload source, so broader validation on additional systems, workloads, and deployment environments is needed.

6 Conclusion

This paper presented a DSPL–DRL approach for self-adaptation in microservice systems, where the feature model defines the valid configuration space and a DDQN agent learns workload-aware reconfiguration decisions using a PPM-based surrogate environment. Using the CaseStudy-A case study, the results showed that the approach achieved near-oracle decision quality, remained competitive with a budgeted GA baseline under trace-driven emulation, and enabled very fast runtime decisions. Future work will focus on live online deployment, evaluation on additional MSS and workload traces, richer QoS and cost metrics, and robustness under workload drift and PPM updates.

Disclosure of Interests The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Ahmad, H., Treude, C., Wagner, M., Szabo, C.: Towards resource-efficient reactive and proactive auto-scaling for microservice architectures. *Journal of Systems and Software* **225**, 112390 (2025)
2. Alf  rez, G.H., Pelechano, V., Mazo, R., Salinesi, C., Diaz, D.: Dynamic adaptation of service compositions with variability models. *Journal of Systems and Software* **91**, 24–47 (2014)
3. Arlitt, M., Jin, T.: A workload characterization study of the 1998 world cup web site. *IEEE network* **14**(3), 30–37 (2000)
4. Authors, A.: Title omitted for double-blind review (2024), earlier DSPL-based study; reference anonymized for double-blind review
5. Authors, A.: Titles omitted for double-blind review (2024–2026), related publications anonymized for double-blind review
6. Chen, T., Guestrin, C.: Xgboost: A scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. pp. 785–794 (2016). <https://doi.org/10.1145/2939672.2939785>
7. De Sanctis, M., Muccini, H., Vaidhyanathan, K.: Data-driven adaptation in microservice-based iot architectures. In: *2020 IEEE International Conference on Software Architecture Companion (ICSA-C)*. pp. 59–62. IEEE (2020)
8. De Sanctis, M., Muccini, H., Vaidhyanathan, K.: A user-driven adaptation approach for microservice-based iot applications. In: *Proceedings of the 11th International Conference on the Internet of Things*. pp. 48–56 (2021)
9. Grafana Labs: K6 . <https://grafana.com/docs/k6/latest/>, accessed: 2026-02-14
10. Grebhahn, A., Siegmund, N., Apel, S.: Predicting performance of software configurations: There is no silver bullet. *arXiv preprint arXiv:1911.12643* (2019)
11. Hallsteinsen, S., Hinchey, M., Park, S., Schmid, K.: Dynamic software product lines. *Computer* **41**(4), 93–95 (2008)
12. He, X., Tu, Z., Xu, X., Wang, Z.: Programming framework and infrastructure for self-adaptation and optimized evolution method for microservice systems in cloud-edge environments. *Future Generation Computer Systems* **118**, 263–281 (2021)
13. Johansen, M.F., Haugen,   ., Fleurey, F.: An algorithm for generating t-wise covering arrays from large feature models. In: *Proceedings of the 16th International Software Product Line Conference-Volume 1*. pp. 46–55 (2012)
14. Lalanda, P., McCann, J.A., Diaconescu, A.: *Autonomic Computing: Principles, Design and Implementation*. Springer, London (2013). <https://doi.org/10.1007/978-1-4471-5007-7>
15. Liu, P., Mao, X., Zhang, S., Hou, F.: Towards reference architecture for a multi-layer controlled self-adaptive microservice system. In: *SEKE*. pp. 236–235 (2018)
16. Metzger, A., Quinton, C., Mann, Z.  ., Baresi, L., Pohl, K.: Feature model-guided online reinforcement learning for self-adaptive services. In: *International Conference on Service-Oriented Computing*. pp. 269–286. Springer (2020)
17. Mousa, A., Bentahar, J., Alam, O.: Context-aware composite saas using feature model. *Future Generation Computer Systems* **99**, 376–390 (2019)
18. Pimentel, E., Pereira, W., Maia, P.H.M., Cort  s, M.I., et al.: Self-adaptive microservice-based systems-landscape and research opportunities. In: *2021 International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. pp. 167–178. IEEE (2021)
19. Rodrigues-Filho, R., Sene Jr, I., Porter, B., Bittencourt, L.F., Kon, F., Costa, F.M.: Exploring emergent microservice evolution in elastic deployment environments. *Journal of Systems and Software* **219**, 112252 (2025)

20. Rossi, F., Cardellini, V., Presti, F.L., Nardelli, M.: Dynamic multi-metric thresholds for scaling applications using reinforcement learning. *IEEE Transactions on Cloud Computing* **11**(2), 1807–1821 (2022)
21. Sampaio, A.R., Rubin, J., Beschastnikh, I., Rosa, N.S.: Improving microservice-based applications with runtime placement adaptation. *Journal of Internet Services and Applications* **10**(1), 1–30 (2019)
22. Sat4j: A satisfiability library. <https://www.sat4j.org/>, accessed: 2025-09-24
23. Siegmund, N., Kolesnikov, S.S., Kästner, C., Apel, S., Batory, D., Rosenmüller, M., Saake, G.: Predicting performance via automated feature-interaction detection. In: 2012 34th International Conference on Software Engineering (ICSE). pp. 167–177. IEEE (2012)
24. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. MIT Press, Cambridge, MA, 2 edn. (2018)
25. Weyns, D., Gerostathopoulos, I., Vogel, T., Andersson, J., Calinescu, R., Schmerl, B.R., Petke, J., Giese, H., Nitto, E.D., Lopes, A., et al.: Self-adaptation in industry: A survey. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* **18**(3), 1–48 (2023). <https://doi.org/10.1145/3589227>
26. Xie, R., Ma, L., Zhang, S., Xia, F., Lin, L.: Reweighting clicks with dwell time in recommendation. In: Companion Proceedings of the ACM Web Conference 2023. pp. 341–345 (2023)
27. Zhang, S., Zhang, M., Ni, L., Liu, P.: A multi-level self-adaptation approach for microservice systems. In: 2019 IEEE 4th International Conference on Cloud Computing and Big Data Analysis (ICCCBDA). pp. 498–502. IEEE (2019)